Eric Romero
January 26, 2026
CSC 445

Homework 1: Write-up

Algorithm Description:

We are given a sorted list of unknown length, accessible only through a constant-time get(index) operation that returns either the element at the given index or null if the index is out of bounds. Since the size of the list is unknown, a standard binary search cannot be applied directly. The goal is to minimize the number of get calls while correctly locating a target element.

Sub-Goal 1: Determine Search Bounds

We begin by identifying an interval that must contain the target if it exists. Starting from index 1, we repeatedly double the index (1, 2, 4, 8, …), querying get(i) at each step. This process continues until either:

- get(i) returns null, indicating that index i is beyond the end of the list.
- get(i) returns a value greater than or equal to the target.

If the algorithm stops at index i, then if the target exists, it must lie within the interval [i/2, i].

Sub-Goal 2: Binary Search Within the Bounds

Once the interval [i/2, i] is established, we perform a standard binary search within this range. During the search:

- If get(mid) returns null, the index mid is out of bounds and is treated as containing a value greater than the target.
- If get(mid) is less than the target, the search continues in the right half.
- If get(mid) is greater than the target, the search continues in the left half.
- If get(mid) equals the target, the index mid is returned.

If the search interval is exhausted without finding the target, the algorithm returns -1.

Correctness Justification:

We show that the algorithm always returns a valid index containing the target if it exists, and -1 otherwise.

Sub-Goal 1 Correctness

The list is sorted in ascending order. The algorithm examines indices that double at each step until reaching an index i such that get(i) is null or get(i) ≥ target.

- If get(i) is null, then index i lies beyond the end of the list, so any valid position containing the target must be less than i.

- If get(i) ≥ target, then due to the sorted order, the target cannot appear at any index greater than i.

In both cases, the previous index i/2 was examined and satisfied get(i/2) < target. Therefore, if the target exists, it must be located within the interval [i/2, i].

Sub-Goal #2:

Binary search is correct on any finite sorted interval. Since Sub-Goal 1 guarantees that the target (if present) lies within the interval [i/2, i], binary search will correctly locate and return an index containing the target, or determine that it does not exist and return -1.

Since Sub-Goal 1 correctly identifies a search interval containing the target if it exists, and Sub-Goal 2 correctly searches that interval, the overall algorithm is correct.

Complexity Analysis:

Let x be the position of the target in the list if it exists, or the position where it would be inserted to maintain sorted order if it does not exist.

Sub-Goal #1:

The indices examined are 1, 2, 4, … 2^k until reaching an index such that x <= 2^k. The smallest such k satisfies ceil($\log_2 x$). Therefore, Sub-Goal 1 performs as O(log x) calls to ‘get’.

Sub-Goal #2:

The search interval is [i/2, i], whose size is at most i/2 + 1 = O(x). Binary search on this interval requires O(log x) additional calls to ‘get’.

Total Time Complexity:

O(log x)

Total Space Complexity:

O(1)